

CARRERA:

TECNICO SUPERIOR EN DESARROLLO DE SOFTWARE

DESARROLLO CURRICULAR ÁULICO DEL MÓDULO / TALLER / ASIGNATURA /

TALLER DE DISEÑO DE SOFTWAREN II



Curso: Primer Año

Año: 2026

Régimen: ANUAL

TURNO: Tarde / Noche

HORAS: Tarde (4 hs) / Noche (4 hs)

Modalidad: Presencial / SemiPresencial / Virtual

PROF. ING. DANIEL MALDONADO

SAN FERNANDO DEL VALLE DE CATAMARCA,

FIRMA Y ACLARACIÓN DEL PERSONAL QUE LO RECIBE:



2- SINTESIS EXPLICATIVA

La Industria de la Ingeniería del Software y el desarrollo de aplicaciones es la actividad económica que mayor valor agregado genera. Todos los países desarrollados del mundo tienen como objetivo fundamental promover esta actividad económica que se basa en algo tan rico como son los recursos humanos capacitados y la actividad netamente intelectual.

Argentina, a través de diferentes planes nacionales y gubernamentales, tiene la intención de promover la Industria del Software brindando capacitaciones, líneas de créditos flexibles, leyes nacionales y acuerdos internacionales. Sin lugar a dudas, la Industria del Software marcará en los próximos años la diferencia entre países desarrollados y subdesarrollados.

Los Técnicos en Desarrollo de Software tienen un gran campo de acción laboral no tan solo en el presente sino también en el futuro. Es imprescindible que los futuros egresados conozcan profundamente cómo desarrollar software, cómo realizar programas fácilmente adaptables con código reutilizable, que conozcan un lenguaje de programación profesional que les permita integrarse al mundo de la Ingeniería del Software y que puedan integrarse a departamentos y organizaciones que producen soluciones informáticas.

El enfoque pedagógico de esta materia está centrado de manera exclusiva en JavaScript como lenguaje de programación profesional. JavaScript es hoy uno de los lenguajes más demandados de la industria — permite desarrollar tanto el Frontend como el Backend de una aplicación, lo que convierte al alumno en un desarrollador Full Stack con un único lenguaje. Esta decisión pedagógica es deliberada: el alumno no aprende un lenguaje de juguete para luego tener que aprender otro — aprende desde el primer día con una herramienta real que usará durante toda su carrera profesional.

HTML y CSS son abordados de manera introductoria como contexto necesario para comprender el entorno donde se ejecuta JavaScript, dado que constituyen contenido específico de otras materias de la carrera. El foco de esta materia es la programación — la lógica, las estructuras, la resolución de problemas y la construcción de soluciones reales.

La metodología es la del taller — el alumno aprende programando. Las clases son prácticas, los ejercicios son graduales y las guías de trabajos prácticos están diseñadas para que el alumno programe de manera autónoma fuera del horario de clase, consolidando el conocimiento a través de la práctica diaria y sostenida. Los enunciados utilizan contextos reales y situaciones cotidianas del entorno argentino, lo que facilita la comprensión y hace que el aprendizaje sea significativo desde el primer ejercicio.

Finalmente, en el contexto actual donde la Inteligencia Artificial forma parte del ecosistema de desarrollo de software, contar con una base sólida de programación se vuelve más importante que nunca. El profesional que comprende los fundamentos puede utilizar las herramientas de IA como un multiplicador de su capacidad — puede evaluar el código que genera, detectar errores, adaptarlo



a su problema y tomar decisiones fundamentadas. El que no tiene esa base queda a merced de lo que la IA produce sin poder entenderlo ni controlarlo. Por eso esta materia no enseña atajos — enseña a pensar, a programar y a comprender, que es exactamente lo que potencia el uso inteligente de cualquier herramienta, incluyendo la Inteligencia Artificial.

3- a- Objetivos Generales

- Brindar una introducción sólida a las herramientas metodológicas necesarias para el desarrollo de software mediante el paradigma de la Programación Modular, utilizando JavaScript como lenguaje de programación profesional.
- Lograr que el estudiante adquiera aptitud en la resolución de problemas a través del desarrollo de algoritmos, la interpretación de enunciados y la codificación de soluciones en JavaScript.
- Que el estudiante logre autonomía y pueda explorar en forma independiente las posibilidades que ofrece JavaScript como lenguaje profesional, tanto en el Frontend como en el Backend.
- Que el estudiante pueda evaluar el resultado de sus soluciones utilizando su capacidad reflexiva y crítica, comprendiendo que un problema puede resolverse de múltiples formas y que siempre existe una solución más eficiente.
- Que el alumno aprenda el oficio de la programación — que tenga su mente abierta a los cambios, que busque las mejores soluciones y que entienda que la práctica diaria y sostenida es el único camino hacia la excelencia.
- Que el alumno logre integrarse a las organizaciones donde desempeñará su función e interprete los alcances de un profesional dedicado al desarrollo de software en el contexto actual de la industria.
- Que el alumno reconozca las estructuras de datos existentes, los algoritmos que permiten la manipulación de datos y cuándo y cómo utilizar cada estructura según el problema a resolver.
- Que el alumno comprenda que en el contexto actual de la Inteligencia Artificial, una base sólida de programación es la que permite utilizar estas herramientas con criterio, evaluando y comprendiendo lo que generan.

a- Objetivos Específicos

- Al finalizar el cursado el alumno comprenderá el entorno de la Web — cómo funciona una página, qué rol cumple JavaScript dentro de las tres tecnologías fundacionales y cómo configurar su entorno de trabajo profesional con VS Code y Live Server.
- El alumno dominará los fundamentos de JavaScript — variables, tipos de datos, operadores, template literals y el concepto de scope — como base necesaria para todo lo que viene después.
- El alumno será capaz de resolver problemas utilizando estructuras de control — condicionales y repetitivas — comprendiendo cuándo y cómo aplicar cada estructura según la naturaleza del problema.
- El alumno comprenderá el concepto de programación modular y podrá definir, invocar y organizar funciones correctamente, aplicando el concepto de caja negra, scope, callbacks y funciones de orden superior.



- El alumno podrá construir aplicaciones con interfaz visual utilizando el manejo básico del DOM — capturando elementos, respondiendo a eventos, mostrando resultados y organizando el código en módulos con export e import.
- Al finalizar el cursado el alumno tendrá los conocimientos necesarios para trabajar con arrays y objetos literales, aplicando métodos de orden superior como forEach, map, filter, find, some, every y reduce sobre estructuras de datos reales.
- El alumno será capaz de interpretar enunciados, codificar soluciones y realizar pruebas de sus programas de manera autónoma, desarrollando criterio propio para evaluar la calidad y eficiencia de su código.
- El alumno comprenderá que los conocimientos adquiridos en esta materia son la base sobre la cual se construirá el desarrollo Backend con Node.js en el segundo año de la carrera.

4- Contenidos del espacio curricular módulo/taller/asignatura

- Contenidos Conceptuales

NÚCLEO TEMÁTICO I — Repaso e Integración de 1° año

Repaso intensivo de los contenidos de 1° año. El objetivo es nivelar, refrescar y consolidar los fundamentos antes de avanzar hacia los contenidos de 2° año.

- Repaso de variables y constantes — let, const, tipos de datos, scope de bloque.
- Repaso de operadores — aritméticos, de asignación, comparación y lógicos.
- Repaso de template literals — backticks e interpolación con \${}.
- Repaso de estructuras condicionales — if/else, operador ternario, switch.
- Repaso de estructuras repetitivas — for, while. Contadores y acumuladores.
- Recorrido de strings carácter por carácter con ciclo for.
- Repaso de funciones — declaración, expresión, arrow functions, parámetros, retorno, valores por defecto.
- Repaso del concepto de caja negra — funciones puras.
- Repaso de funciones de orden superior y funciones callback.
- Repaso de arrays — creación, acceso por índice, recorrido con for.
- Repaso de métodos de orden superior — forEach, map, filter, find, some, every, reduce.
- Repaso de objetos literales — propiedades, acceso con punto y corchetes.
- Repaso de arrays de objetos literales.
- Repaso de destructuring de arrays y objetos.
- Repaso de Spread operator y Rest operator.
- Repaso de módulos — export e import.
- Repaso de DOM básico — querySelector, onclick, textContent, innerHTML.

NÚCLEO TEMÁTICO II — JavaScript Avanzado del lado del cliente

Patrón de diseño MVC — Modelo Vista Controlador



- Concepto de arquitectura de software — separación de responsabilidades.
- El archivo modelo.js — fuente de verdad, datos y lógica de negocio.
- El archivo vista.html — estructura visual e interfaz de usuario.
- El archivo controlador.js — captura de eventos y delegación a funciones.

Manipulación dinámica del DOM

- window.onload — ejecución diferida hasta que la página termina de renderizar.
- querySelector() — captura de elementos del DOM.
- Creación dinámica de elementos — createElement, appendChild, insertBefore.
- Eliminación de elementos — removeChild, remove.
- Modificación de atributos — setAttribute, getAttribute.
- Propiedades textContent e innerHTML — diferencias y uso correcto.
- Renderización dinámica de listas y tablas desde arrays de objetos.
- Estado de la aplicación — representación como objeto literal y sincronización con el DOM.
- classList avanzado — add(), remove(), toggle(), contains(), replace().

Eventos

- Evento onclick y oninput — acciones sobre elementos en tiempo real.
- Evento onchange — detección de cambios al salir del input.
- Delegación de eventos avanzada.
- Objeto event — event.target, event.preventDefault().

JSON

- Qué es JSON — JavaScript Object Notation.
- JSON.stringify() — convertir un objeto a string JSON.
- JSON.parse() — convertir un string JSON a objeto.
- Uso práctico: persistir y recuperar el estado de la aplicación.

Almacenamiento en el navegador

- localStorage — setItem(), getItem(), removeItem() — persistencia permanente.
- sessionStorage — diferencias con localStorage, persistencia por sesión.
- Cookies e IndexedDB — concepto general.

Asincronismo

- Qué es el asincronismo y por qué existe en JavaScript.
- El Event Loop — cómo funciona JavaScript por dentro.
- Promesas — concepto general, mención introductoria.
- async / await — sintaxis moderna para manejar asincronismo.
- try / catch aplicado a código asíncrono.
- fetch API — consumo de APIs externas desde el navegador.
- Ciclo completo: fetch → await → .json() → renderizar datos en el DOM.
- Consumo de APIs públicas reales — práctica con datos reales.

Módulos

- export de funciones, constantes y objetos.
- import desde el controlador.



- Atributo type='module' en el script HTML.

NÚCLEO TEMÁTICO III — Bases de Datos Relacionales y SQL con PostgreSQL

Conceptos fundamentales

- Qué es un Sistema de Gestión de Bases de Datos Relacional (SGBDR).
- PostgreSQL — instalación y presentación del motor de base de datos.
- pgAdmin — instalación y uso de la interfaz gráfica para administrar PostgreSQL.
- Tablas, filas y columnas — estructura fundamental de una base de datos relacional.
- Tipos de datos en SQL — VARCHAR, INTEGER, BOOLEAN, DATE, NUMERIC.
- Clave primaria (Primary Key) — identificación única de registros.
- Clave foránea (Foreign Key) — relación entre tablas.
- Índices únicos — garantizar unicidad en campos específicos.
- Valores autoincrementales — SERIAL en PostgreSQL.
- Restricciones — NOT NULL, UNIQUE, DEFAULT.

DDL — Lenguaje de Definición de Datos

- CREATE DATABASE — crear una base de datos.
- CREATE TABLE — crear tablas con sus columnas y restricciones.
- ALTER TABLE — modificar la estructura de una tabla existente.
- DROP TABLE — eliminar una tabla.

DML — Lenguaje de Manipulación de Datos

- INSERT INTO — insertar registros en una tabla.
- UPDATE — modificar registros existentes.
- DELETE — eliminar registros.
- SELECT — consultar datos.
- Cláusulas: WHERE, ORDER BY, LIMIT.

Consultas de combinación

- INNER JOIN — registros que coinciden en ambas tablas.
- LEFT JOIN — todos los registros de la tabla izquierda.
- RIGHT JOIN — todos los registros de la tabla derecha.
- Combinación de múltiples tablas en una sola consulta.

NÚCLEO TEMÁTICO IV — Node.js y Desarrollo Backend

Instalación y configuración del entorno

- Instalación de Node.js.
- npm — Node Package Manager — qué es y para qué sirve.
- Inicialización de un proyecto Node.js — npm init.
- Instalación de paquetes — npm install.



- El archivo package.json — estructura y propiedades.
- El archivo package-lock.json y la carpeta node_modules.
- El archivo .gitignore — qué ignorar en el proyecto.
- Variables de entorno — archivo .env y paquete dotenv.

Servidor con Express

- Qué es Express — framework web para Node.js.
- Instalación de Express.
- Crear y arrancar un servidor — app.listen().
- El puerto donde escucha el servidor.
- Rutas — app.get(), app.post(), app.put(), app.delete(), app.patch().
- Equivalencia con operaciones SQL: GET→SELECT, POST→INSERT, PUT→UPDATE, DELETE→DELETE, PATCH→UPDATE parcial.

Request y Response

- El objeto req — datos que llegan al servidor.
- El objeto res — datos que devuelve el servidor.
- res.json() — devolver datos en formato JSON.
- res.status() — códigos de estado HTTP — 200, 201, 400, 404, 500.
- Parámetros de ruta — req.params.
- Parámetros de consulta — req.query.
- Cuerpo de la petición — req.body.
- Middleware — express.json() para parsear el body.

Conexión a PostgreSQL desde Node.js

- Instalación del paquete pg — node-postgres.
- Configuración de la conexión — host, port, database, user, password.
- Pool de conexiones.
- Ejecutar consultas SQL desde Node.js — pool.query().
- Manejo de errores en las consultas — try/catch.

Arquitectura del proyecto Backend

- Separación en capas — rutas, controladores, modelos.
- Archivos de rutas — router.
- Nodemon — reinicio automático del servidor ante cambios en el código.
- Postman — prueba y documentación de endpoints REST.
- express.static() — servir el frontend estático desde Node.js, todo en una sola carpeta y un solo servidor.
- CORS — Cross-Origin Resource Sharing — qué es y cuándo ocurre el problema.
- Instalación del paquete cors con npm install cors.
- Configuración de origen permitido — restringir el acceso únicamente al dominio propio en modo testing: http://127.0.0.1:5500.

Integración Full Stack — Frontend con su propia API

- fetch desde el frontend hacia la propia API.
- Insertar datos — formulario HTML → fetch POST → INSERT en PostgreSQL.



- Recuperar datos — fetch GET → SELECT → renderizar en el DOM.
- Modificar datos — formulario de edición → fetch PUT → UPDATE.
- Eliminar datos — botón eliminar → fetch DELETE → DELETE.
- CRUD completo — Create, Read, Update, Delete sobre la propia API.

NÚCLEO TEMÁTICO V — Proyecto Final Integrador

El Proyecto Final Integrador es la instancia de cierre del año donde el alumno demuestra que puede integrar y aplicar de manera autónoma todos los contenidos vistos durante los dos años de la carrera. No es un ejercicio guiado — es un sistema real pensado, diseñado e implementado por el propio alumno.

Definición y diseño del sistema

- El alumno define una idea propia de sistema — original y significativa.
- El sistema debe contemplar un diseño de base de datos de entre 7 y 8 tablas relacionadas.
- Diseño del modelo de datos — diagrama de tablas, relaciones, claves primarias y foráneas.
- Definición de los endpoints necesarios — qué operaciones expone la API.
- Definición de las vistas necesarias — qué pantallas tendrá el frontend.

Implementación — Base de datos

- Creación de la base de datos en PostgreSQL.
- Creación de tablas con sus relaciones, restricciones y tipos de datos correctos.
- Carga de datos iniciales de prueba.

Implementación — Backend

- Proyecto Node.js con Express correctamente estructurado.
- Separación en capas — rutas, controladores, modelos.
- Endpoints REST conectados a PostgreSQL — al menos las operaciones más representativas del sistema.
- Manejo de errores — try/catch y códigos de estado HTTP correctos.
- Variables de entorno con dotenv.
- Frontend servido desde Node.js con express.static().
- CORS configurado para el origen permitido.

Implementación — Frontend

- Interfaz HTML conectada a la propia API mediante fetch con async/await.
- Patrón MVC — vista, controlador y modelo separados.
- Estado de la aplicación representado como objeto literal.
- Renderización dinámica de datos en el DOM.
- CRUD completo o parcial según la naturaleza del sistema.

Evaluación y presentación

- No se requiere implementar el sistema completo — se evalúa la parte más representativa e importante.



- El alumno presenta y defiende el proyecto ante el docente.
- Se evalúa la comprensión del sistema, la calidad del código y la integración de los conceptos.
- Se valora la originalidad de la idea, la coherencia del modelo de datos y la arquitectura del proyecto.

- **Contenidos Procedimentales**

- Análisis e interpretación de enunciados y problemas típicos que se presentan en el ámbito del desarrollo de software.
- Transformación de enunciados en código JavaScript — del problema planteado a la solución codificada.
- Utilización de la consola del navegador como herramienta de desarrollo, prueba y depuración de programas.
- Escritura, ejecución y corrección de programas JavaScript en Visual Studio Code con Live Server.
- Resolución de los conflictos y errores más comunes que se presentan en el oficio del programador — lectura e interpretación de mensajes de error.
- Realización de pruebas completas de los programas desarrollados — verificación de resultados con diferentes valores de entrada.
- Organización del código en módulos — separación de responsabilidades entre vista, controlador y modelo.
- Aplicación del patrón de diseño de caja negra en la construcción de funciones — entradas, procesamiento y salida sin efectos externos.
- Construcción progresiva de proyectos que integren todos los contenidos del año — desde ejercicios simples de consola hasta pequeñas aplicaciones con interfaz visual.
- Uso de guías de trabajos prácticos extensas con ejercicios graduados — práctica autónoma diaria fuera del horario de clase.
- Búsqueda, lectura e interpretación de documentación técnica — MDN Web Docs, javascript.info — como habilidad profesional fundamental.
- Utilización de agentes de Inteligencia Artificial como herramienta de aprendizaje y consulta — búsqueda de explicaciones, ejemplos y resolución de dudas, desarrollando el criterio para evaluar y comprender las respuestas obtenidas antes de aplicarlas.

- **Contenidos Actitudinales (generales para todas las unidades)**

- Actitud proactiva ante los desafíos de programación — el alumno busca la solución antes de rendirse.
- Constancia y perseverancia para encontrar la solución a los problemas — entendiendo que el error es parte natural del proceso de aprendizaje y no un fracaso.
- Mente abierta para entender y comprender que un mismo problema puede resolverse de diferentes formas — no existe una única solución correcta.
- Actitud de permanente superación — el alumno debe saber que algo que ya resolvió puede ser mejorado, haciendo sus soluciones más eficaces y eficientes.



- Deseo de superación y capacitación continua — JavaScript es un lenguaje en constante evolución que exige actualización permanente.
- Gran capacidad de adaptación — la industria del software se caracteriza por la constante actualización tecnológica y el alumno debe estar preparado para aprender nuevas herramientas a lo largo de toda su carrera.
- Respeto por las buenas prácticas de programación — código legible, comentado, organizado y con nombres descriptivos.
- Responsabilidad y compromiso con la entrega de los trabajos prácticos en tiempo y forma.
- Actitud colaborativa — disposición para compartir conocimiento, ayudar a los compañeros y aprender de los demás.
- Honestidad intelectual — el alumno presenta trabajo propio, comprende lo que entrega y puede defenderlo y explicarlo.
- Criterio reflexivo ante el uso de la Inteligencia Artificial — el alumno utiliza estas herramientas como apoyo al aprendizaje pero asume la responsabilidad de comprender, evaluar y validar todo lo que aplica en su código.

5- Orientaciones Metodológicas

Al ser una materia en formato TALLER, los alumnos adquieren el conocimiento a través de la práctica guiada y asistida por el docente. El enfoque está centrado exclusivamente en JavaScript como lenguaje de programación profesional, con ejercitación intensiva y progresiva a través de guías de trabajos prácticos que van desde ejercicios simples hasta problemas de complejidad creciente.

Las clases son presenciales y prácticas — el docente narra, explica y codifica en tiempo real frente a los alumnos, construyendo los ejemplos desde cero y explicando cada decisión de diseño. El alumno observa, participa, pregunta y luego replica y extiende lo visto en sus propios ejercicios.

Las guías de trabajos prácticos están diseñadas para que el alumno programe de manera autónoma fuera del horario de clase, con el objetivo de sostener una práctica diaria de al menos una hora. Los enunciados utilizan contextos reales y situaciones cotidianas del entorno argentino — comercios, servicios públicos, instituciones bancarias, programas gubernamentales — lo que facilita la comprensión y hace que el aprendizaje sea significativo desde el primer ejercicio.

Los recursos metodológicos utilizados son los siguientes:

- Clases prácticas con resolución de ejercicios en tiempo real frente a los alumnos.
- Clases de laboratorio — programación en computadora con VS Code y Live Server.
- Guías de trabajos prácticos extensas con ejercicios graduados de menor a mayor complejidad.
- Trabajos individuales y en grupo.
- Material de apoyo disponible en la plataforma Google Classroom.



- Uso de agentes de Inteligencia Artificial como herramienta de consulta y aprendizaje autónomo, desarrollando en el alumno el criterio para evaluar y comprender las respuestas obtenidas.
- Evaluación sumativa por núcleo temático — parcial práctico al finalizar cada unidad.

6- Tiempo

Explicitar el tiempo aproximado para el desarrollo de cada unidad

Estos son los tiempos aproximados y estimados por Unidad para Cada Turno.

CUATRIMESTRE	NÚCLEO TEMÁTICO	CLASES	HS CÁTEDRA
1°	Núcleo I — Repaso e Integración de 1° año	4	8
1°	Núcleo II — JavaScript Avanzado del lado del cliente	10	20
1°	Núcleo III — Bases de Datos Relacionales y SQL con PostgreSQL	8	16
2°	Núcleo IV — Node.js y Desarrollo Backend	12	24
2°	Núcleo V — Proyecto Final Integrador	8	16
	TOTAL	42	84



7 - Guías de Trabajos Prácticos

Al ser una materia en formato TALLER, los alumnos deben conseguir el conocimiento a través de la práctica guiada y asistida por parte del docente y de todo el material virtual que el alumno tiene a disposición.

7.1) Material Disponible En línea

- Correo Electrónico Institucional
- Plataforma Educativa Classroom donde tiene contenido actualizado de la Materia
- Para Todas las Modalidades Presencial/Semipresencial/Virtual existe una guía de Trabajos Prácticos extensa que permite al alumno ejercitar y adquirir los conocimientos teóricos y prácticos establecidos en el presente diseño curricular. Sobre cada punto existe un video tutorial online disponible para que el alumno utilice como práctica guiada, corrija sus errores y pueda observar la manera correcta de realizar los ejercicios.
- Exclusivamente para la Modalidad Semipresencial/Virtual el profesor cuenta con un grupo de comunicación What'sapp para una comunicación directa y fluida entre el grupo de alumnos y el profesor para asistirlo en todo momento.

7.2) Guías de Trabajos Prácticos (Modalidades Presencial/SemiPresencial/Virtual)

Para el Desarrollo completo de la materia, la misma cuenta con guías de Trabajos prácticos organizados en módulos, cada módulo incluye una temática en especial dentro del desarrollo de aplicaciones móviles. Cada módulo puede estar dividido en partes que tocan una temática en especial tal como se muestra a continuación en el siguiente esquema. El primer cuatrimestre cuenta con 4 Módulos y el objetivo de cada módulo esta descrito en el siguiente cuadro.

El segundo cuatrimestre se encuentra en proceso de Construcción.



7.3) Ventajas del Material Audio Visual En Línea

Aprendizaje significativo: Tu objetivo principal como docente es que los estudiantes tengan un aprendizaje significativo, es decir, que asimilen la información y la puedan poner en práctica. La educación virtual da acceso a los estudiantes a repasar el contenido cuantas veces lo consideren necesario y desarrollar las actividades a su propio ritmo. La ventaja frente a la educación tradicional es que el estudiante tendrá la posibilidad de repasar tus sesiones sin que esto interrumpa el proceso de los demás estudiantes, lo que a largo plazo será beneficioso tanto para él como para sus compañeros.

Accesibilidad: Permite a Estudiantes acceder al contenido desde cualquier lugar de la provincia disponiendo de una conexión a internet.

Comunicación asincrónica: En la educación virtual, la comunicación entre el estudiante puede ser asincrónica, es decir que la interacción a través de medios digitales se desarrolla en horarios diferidos, evitando que se provoquen retrasos en el proceso de aprendizaje.

Trabajo Colaborativo: La educación virtual facilita el trabajo colaborativo, el acceso a chats, debates y prácticas en las plataformas, enriquecen los conocimientos.

Evaluación Permanente: Un sistema de evaluación continua, en el que el estudiante es consciente y responsable de su proceso de aprendizaje, a la vez que tiene mecanismos alternativos de evaluación diferentes al examen tradicional.



- 7- **Evaluación:** Ver anexo de sistema de evaluación y promoción de asignaturas proporcionadas y estandarizadas por el INSTITUTO SUPERIOR GENERAL SAN MARTIN.

CLASIFICACIÓN	
Según el momento	Según el alcance
Evaluación Inicial	Realizaremos un diagnóstico para establecer los contenidos previos que traen los alumnos de la materia anterior y en función de ello determinar el punto de partida de los contenidos de esta materia.
Evaluación de Proceso	Durante el transcurso del año lectivo, se evaluará al alumno mediante trabajos prácticos, talleres de dos modalidades. <u>Trabajos Prácticos en Clases o Taller:</u> donde el profesor proporcionará una consigna y el alumno en forma individual deberá desarrollar las tareas proporcionadas por el Profesor. Tiempo máximo para desarrollar es de un módulo (máximo dos horas cátedra). <u>Prácticos a desarrollar en Casa:</u> donde el profesor proporcionará una consigna y el alumno en forma individual o grupal deberá desarrollar las tareas proporcionadas por el profesor. Tiempo máximo para desarrollar la actividad es de una semana.
Evaluación Final	<u>Trabajo Práctico Final:</u> que será INDIVIDUAL con la intención que se ponga en práctica todo lo estudiado, practicado y analizado durante el año. Este trabajo práctico final es Obligatorio y requisito fundamental para regularizar la materia.



Criterios de evaluación:

Los puntos que a continuación describo son los más importantes que evalúo durante el cuatrimestre y por el cual determino el nivel de conocimiento que el educando adquirió sobre la materia.

- **Asistencia del alumno y continuidad en las clases:** Es de suma importancia la asistencia y continuidad del alumno a las clases dictadas por el profesor. Dado que los contenidos vertidos en clases son un cúmulo de conocimientos específicos de la materia como así también experiencias profesionales del docente/profesional vinculados directamente. Mucho del material entregado en clases como las experiencias y analogías utilizadas por el profesor no son material que puedan encontrarse fácilmente en un libro, ni en internet, ni en videos. La formación de un futuro profesional técnico informático requiere de docentes que puedan entregar a sus educandos un no tan solo el contenido específico de las Unidades sino también mostrar en el campo real la utilización de los conceptos, las ventajas y/o desventajas de utilizar un método u otro, etc.
- **Participación en Clases:** Iniciar a los alumnos en el Terreno de la Programación es un proceso de transformación de los educandos. Es decir aquí los alumnos deberán diseñar lógicamente procesos, pensar en soluciones alternativas a las presentadas por el profesor en clases, deberán cuestionar y cuestionarse entre ellos las soluciones que se plantean, deberán discutir e ir formando criterio, pensando y acomodando sus ideas, ordenando mentalmente los pasos a seguir. Es todo un proceso de Transformación, de cambio que implica también resistencia a incorporar esos conceptos. Es importante que el alumno en clases participe, discuta, refleje todas sus dudas e inquietudes individuales y colectivas.
- **Presentación de los Trabajos Prácticos:** Dentro de este punto el profesor evaluará la presentación en término cumpliendo con las fechas establecidas en el cronograma como así también la prolijidad, eficacia y eficiencia de los programas presentados.
- **Interés del Alumno por la Materia:** Si bien es un punto difícil de mensurar; el profesor observa el interés del alumno por aprender los conceptos vertidos en clases como así también la preocupación, la motivación que tiene por mejorar, superarse ampliando los conocimientos adquiridos en clases. Dado que el contenido de la Materia es un Pilar básico que el alumno tendrá en su futura carrera profesional es requisito fundamental que el Profesor motive al alumno y este refleje interés por la misma.



8- **BIBLIOGRAFÍA BÁSICA:** Realizar detalle de la bibliografía obligatoria con apellido y nombre de autor, obra y **año de edición**.

- Eloquent JavaScript - A Moderne Introduction to Programming
Third Edition
Marijn Haverbeke

BIBLIOGRAFÍA AMPLIATORIA: Realizar detalle de la bibliografía obligatoria con apellido y nombre de autor, obra y **año de edición**.

- JavaScript for Absolute Beginners
Terry MacNavage

BIBLIOGRAFIA PARA EL DOCENTE: Describir cual es el material bibliográfico que utiliza para desarrollar su asignatura.

- JavaScript-The-Definitive-Guide-6th-Edition
The Definitive Guide
David Flanagan